

Bản đồ cây

TÓM TẮT

Đây là bản đồ điều hướng của cả cây: nó nói cây gồm gì, vì sao được chia thành năm phần theo thứ tự đó, và bốn luận điểm sẽ gặp lại ở gần như mọi chương. Đọc xong bạn biết nên bắt đầu từ đâu, chương nào là xương sống phải tự dẫn lại công thức, chương nào chỉ cần biết là có. Nếu chỉ nhớ một điều: *mục tiêu 5 mm / 0,5° không bị giới hạn bởi thuật toán mà bởi ngân sách sai số* — chủ yếu là hình học của constellation, độ chính xác cơ khí của tấm marker, và calibration. Toàn bộ cây được dựng để làm rõ câu đó, để chỉ ra chỗ nó dẫn tới quyết định thiết kế cụ thể, và để chỉ ra khi nào nó hỏng.

Cây này được dựng từ một lộ trình tự soạn, với một mục tiêu hẹp và một bài toán cụ thể: *đưa một robot đi động cập vào dock với sai lệch dưới 5 mm và 0,5°, dùng marker thị giác*. Nó không nhằm liệt kê cho đủ các họ marker đã có, cũng không nhằm là một khảo sát về fiducial marker nói chung. Sự khác biệt giữa hai mục tiêu ấy quyết định toàn bộ cách trình bày phía sau, nên đáng nói rõ ngay từ đầu: mọi chương ở đây đều được viết để trả lời câu hỏi *“cái này ăn bao nhiêu milimét trong ngân sách của tôi”*, chứ không phải câu hỏi *“phương pháp này có gì hay”*.

Luận điểm trung tâm: ngân sách sai số, không phải thuật toán (The Error Budget, Not the Algorithm)

Người mới vào bài toán docking bằng marker gần như luôn đi theo cùng một con đường. Chọn một họ tag — thường là AprilTag vì nó nổi tiếng và có mã nguồn tốt. Dán một tag lên dock. Gọi solvePnP. Nhìn con số pose trả về, thấy nó nhảy, rồi bắt đầu một chuỗi dài những việc mà ai cũng làm: thêm bộ lọc trung bình, thử tag lớn hơn, thử detector khác, thử một thuật toán PnP khác, đọc về lọc Kalman. Chuỗi ấy tiêu tốn nhiều tuần và nó *cải thiện* kết quả, đủ để duy trì cảm giác đang đi đúng hướng. Nhưng nó thường dừng lại ở quanh một, hai độ sai lệch góc, và không ai hiểu vì sao không xuống thấp hơn được.

Cách nhìn đó sai, và nó sai theo một kiểu tổn kém đặc biệt: nó tốn tiền gia công. Bài toán ở đây không phải bài toán thuật toán mà là bài toán **ngân sách sai số** (*error budget*) — cùng loại bài toán mà kỹ sư đo lường làm khi thiết kế một hệ thống đo, và có chuẩn quốc tế riêng cho nó [1]. Sai lệch cuối cùng ở mũi robot là tổng hợp của một chuỗi: nhiều định vị góc trên ảnh, hình học của tập marker, sai số cơ khí của tấm in marker, sai số intrinsic của camera, sai số hand-eye giữa camera và thân robot, và cuối cùng là sai số bám của bộ điều khiển. Mỗi khâu đóng góp một lượng, và *lượng nhỏ nhất trong số đó không quyết định gì cả* — khâu lớn nhất mới quyết định. Thay một detector tốt hơn để giảm nhiễu góc từ 0,2 px xuống 0,1 px là vô nghĩa nếu tấm marker của bạn cong 1 mm.

Điều này không chỉ là một nhận xét đẹp. Nó có một hệ quả thực dụng ngay, và hệ quả ấy đảo ngược thứ tự công việc: *phép tính đầu tiên phải làm không phải là viết code mà là lập ngân sách*. Với các con số hình học của hệ thống dự kiến, ta tính được ngay sai số vị trí và sai số góc kỳ vọng, trước khi mua một con camera nào. Chương 15 làm đúng phép tính đó và nó cho ra một kết luận mà tôi tin là bất ngờ với phần lớn người đọc: *sai số tịnh tiến gần như luôn đạt, còn sai số xoay mới là nút thắt*, và cách chữa nút thắt ấy là hình học — tăng baseline, phá tính đồng phẳng — chứ không phải thuật toán tinh vi hơn.

TRỰC GIÁC

Hình dung chuỗi đo như một sợi dây xích treo vật. Sức chịu của xích bằng sức chịu của mắt yếu nhất, không phải trung bình của các mắt. Bạn có thể thay năm mắt xích bằng thép tốt nhất thế giới, và nếu mắt thứ sáu vẫn là dây thép buộc thì tải trọng cho phép không đổi một gram.

Ngân sách sai số là bản kê khai sức chịu của từng mắt. Việc đầu tiên không phải là cải thiện mắt nào cả, mà là *cân từng mắt* để biết mắt nào yếu nhất. Gần như mọi lần, mắt yếu nhất không nằm ở chỗ người ta nghĩ: nó không phải thuật toán PnP, mà là độ phẳng của tấm nhôm dán tag, hoặc là chuyện constellation của bạn đồng phẳng nên góc gần như không quan sát được.

Và có một mắt xích mà ta hay quên hẳn: pose bạn *thực sự* cần là $T_{\text{dock}}^{\text{base}}$ — từ dock sang thân robot. Marker chỉ là một mắt trung gian. Mỗi phép biến đổi trên chuỗi từ tag tới dock frame, rồi từ camera tới base frame, đều là một mắt xích có sai số riêng, và không mắt nào trong số đó được cải thiện bởi một detector tốt hơn.

Bốn hệ quả kéo theo, và chúng chi phối cả cây (Four Consequences)

Lược điểm trung tâm sinh ra bốn hệ quả cụ thể. Chúng xuất hiện lại ở những chỗ tưởng như không liên quan, và nhận ra chúng là dấu hiệu đã đọc đúng.

Thứ nhất, **translation dễ, rotation khó**. Với các con số điển hình của một bài toán docking, sai số vị trí ngang cỡ phần mười milimét và sai số dọc trục cỡ một milimét — thoải mái trong ngân sách. Còn sai số xoay ước lượng từ *một tag phẳng nhìn gần vuông góc* dễ dàng vượt một độ. Chương 5 dẫn ra vì sao: độ nhạy của phép đo theo góc nghiêng tỉ lệ với chính sin của góc nghiêng ấy, nên ở tư thế fronto-parallel — đúng tư thế mà một robot tiếp cận dock thẳng trục sẽ ở — thông tin về góc gần như biến mất. Đây là một bài toán *observability*, không phải bài toán nhiễu.

Thứ hai, **hình học thắng thuật toán**. Cách chữa nút thắt xoay là dùng nhiều marker đặt xa nhau và *không đồng phẳng*. Sai số góc từ một constellation có baseline B xấp xỉ sai số vị trí ngang chia cho B , nên nó cải thiện tuyến tính theo baseline — một đại lượng bạn mua được bằng cách đặt hai tấm tag cách nhau nửa mét, không phải bằng cách viết thêm code. Phá tính đồng phẳng thì còn làm một việc thứ hai, quan trọng không kém: nó khử tận gốc hiện tượng hai nghiệm của target phẳng (chương 4).

Thứ ba, **cơ khí là sàן sai số**. Nếu vị trí tương đối giữa các tag trong constellation sai 1 mm so với mô hình bạn nạp vào PnP, thì không phép xử lý nào lấy lại được milimét ấy. Tag in giấy dán lên bề mặt cong sai hơn thế. Hệ quả thực hành rất cụ thể và nó nằm ở chương 12: *đừng tin bản vẽ CAD, hãy tự đo lại constellation bằng bundle adjustment* từ nhiều ảnh chụp ở nhiều góc [2]. Đây là bước rẻ và nó thường là bước quyết định giữa đạt và không đạt.

Thứ tư, và đây là mero lớn nhất của cả cây: **docking cần repeatability, không cần accuracy**. Robot không cần biết nó đang ở đâu trong hệ toạ độ tuyệt đối của thế giới; nó cần trở về *đúng chỗ nó đã từng ở*. Hai yêu cầu ấy khác nhau về bản chất và tiêu chuẩn robot công nghiệp phân biệt rõ chúng [3]. Nếu ta ghi lại pose (hoặc thẳng luôn là toạ độ góc trên ảnh) tại vị trí dock đúng, rồi điều khiển để triệt tiêu *sai lệch tương đối* so với bản ghi đó, thì mọi sai số hệ thống — intrinsic sai, kích thước tag sai, hand-eye sai — xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu lẫn nhau. Đây là nội dung của chương 13, và nếu bài toán của bạn đúng là “lập lại một vị trí”, thì đó là con đường ngắn nhất tới 5 mm.

Vì sao chia thành năm phần (The Five Parts)

Năm phần không phải năm chủ đề song song mà là năm tầng câu hỏi nối tiếp. Phần A trả lời *sai số sinh ra ở đâu và lan truyền thế nào*, và nó là phần nền: không có nó thì mọi con số ở Phần C là phép thuật. Phần B trả lời *có những cách nào để giảm từng nguồn sai số, và mỗi cách trả giá bằng gì*. Phần C là chỗ cả cây hội tụ: lập ngân sách bằng số cho một cấu hình cụ thể, rồi kiểm nó bằng Monte Carlo trước khi tốn tiền gia công. Phần D là tầng công cụ — cố ý đặt gần cuối vì nó hết hạn nhanh nhất. Phần E là lộ trình rút gọn.

Bảng 1. Năm phần, câu hỏi mỗi phần trả lời, và mức độ ưu tiên.

	Phần	Câu hỏi nó trả lời	Mức
A	Nguyên lý và toán học	Sai số sinh ra ở đâu, và một pixel nhiễu biến thành bao nhiêu milimét?	cốt lõi
B	Giải thuật và phương pháp	Có những cách nào giảm từng nguồn, và mỗi cách trả giá bằng gì?	cốt lõi
C	Ngân sách sai số	Cấu hình cụ thể của tôi có đạt 5 mm / 0,5° không, và biết trước bằng cách nào?	cốt lõi
D	Framework và công cụ	Ai đã cài sẵn cái gì, và đọc mã nguồn nào để học được nhiều nhất?	công cụ
E	Lộ trình	Nếu chỉ có thời gian cho phần xương sống thì đọc gì, theo thứ tự nào?	điều hướng

Bảng 1 cũng là thứ tự đọc đề xuất cho lần đầu, với một ngoại lệ đáng kể và tôi khuyên dùng nó: *nếu bạn sắp phải quyết định bố trí marker hoặc sắp đặt gia công, hãy đọc chương 15 trước tiên*, ngay cả khi chưa hiểu hết các công thức trong đó. Chương 15 chỉ cần bốn quan hệ tỉ lệ từ chương 5, và nó cho bạn biết ngay bố trí nào có cơ hội đạt mục tiêu. Một buổi chiều ở chương 15 tiết kiệm được vài tuần và vài triệu tiền gia công tám marker sai.

Ba mức ưu tiên, đánh dấu ngay ở tiêu đề chương (Three Priority Levels)

Không phải chương nào cũng đáng đầu tư như nhau. Cây dùng đúng ba mức, lấy từ quy ước sao trong lộ trình gốc (*** cốt lõi, ** quan trọng, * bổ trợ), và mức xuất hiện ngay ở dòng đầu mỗi chương.

Bảng 2. Ba mức ưu tiên và cách đọc tương ứng.

Mức	Nghĩa	Cách đọc tương ứng
● ● ● xương sống	Không hiểu thì không đạt được mục tiêu 5 mm	Đọc kỹ, tự dẫn lại công thức trên giấy , làm ví dụ nhỏ tính tay, trả lời hết câu hỏi tự kiểm
● ● quan trọng	Cần hiểu ý tưởng và biết khi nào dùng	Đọc hiểu cơ chế và điều kiện áp dụng; không cần tự dẫn lại toàn bộ
● bổ trợ	Biết là có, tra khi gặp vấn đề cụ thể	Đọc lướt một lượt để biết trong đó có gì, quay lại khi cần

Phân bố không đều là có chủ ý. Xương sống của cây gồm bảy chương: chương 4 (pose ambiguity), chương 5 (lan truyền sai số), chương 9 (constellation), chương 10 (xử lý ambiguity), chương 12 (calibration), chương 13 (docking control) và chương 15 (ngân sách sai số). Đó là bảy chương

chiếm phần lớn giá trị của cả cây, và chúng cũng chính là lộ trình rút gọn ở Phần E. Đáng chú ý là trong bảy chương ấy, chỉ có hai chương nói về thuật toán xử lý ảnh — điều đó không phải ngẫu nhiên mà là hệ quả trực tiếp của luận điểm trung tâm.

Ba điều mà cây này cố ý nói ngược với thói quen phổ biến (Three Contrarian Claims)

Đáng nêu sớm ba chỗ mà kết luận của cây khác với thứ người ta hay làm, để người đọc biết mình sắp gặp gì.

Một. Không nên trung bình pose từ từng tag. Với nhiều tag nhìn thấy cùng lúc, cách đúng là chạy một bài toán PnP duy nhất trên toàn bộ tập góc, với tọa độ 3D trong hệ dock. Trung bình pose là sai về mặt thống kê (pose sống trên đa tạp, và mỗi pose thành phần có hiệp phương sai rất dị hướng), và nó vứt bỏ đúng thứ ta cần: ràng buộc hình học giữa các tag. Chương 9 giải thích và chương 3 cho nền.

Hai. AprilTag không phải lựa chọn chính xác nhất cho khâu đo tinh. Nó xuất sắc ở khâu *nhận diện và bắt bám từ xa* — bền với che khuất, decode tin cậy, tỉ lệ báo sai cực thấp. Nhưng góc của nó đến từ giao hai đường thẳng khớp trên cạnh, còn góc chessboard đến từ một điểm yên ngựa có cấu trúc đối xứng hai chiều, và cái sau định vị chính xác hơn. Chương 8 bàn chuyện này, và kết luận thực hành là một target lai: AprilTag để bắt bám, vùng chessboard/ChArUco để đo tinh ở cự ly gần.

Ba. Côn dẫn hướng cơ khí ở dock thường là giải pháp rẻ nhất, và nó nên được xét *trước* khi ép thị giác xuống 5 mm. Một chamfer dẫn hướng biến yêu cầu “vào đúng 5 mm” thành “vào trong 15 mm rồi cơ cấu tự dẫn nốt” — một bài toán dễ hơn hẳn một bậc, và lý thuyết của nó đã có từ lâu trong ngành lắp ráp tự động [4]. Chương 13 nói rõ, và đây là chỗ mà một cây tri thức về thị giác phải trung thực nói rằng đôi khi câu trả lời không nằm ở thị giác.

Cách dùng cây (How to Use This Tree)

Mỗi thư mục có một tệp `00-map.tex` nói nhánh đó gồm gì và vì sao chia như vậy; đó là chỗ nên đọc trước khi vào nội dung. Mỗi tài liệu mở đầu bằng một dòng *Phụ thuộc* cho biết cần đọc gì trước, và kết bằng một mục *Kết nối* trỏ sang các nhánh liên quan cùng một câu giải thích quan hệ. Cuối mỗi tài liệu là ba tới bảy câu hỏi tự kiểm; theo lịch ôn của kho, hãy trả lời chúng *trước* khi mở lại bài, sau vài ngày, hai tuần, hai tháng, rồi nửa năm.

Tệp `99-chưa-biết.md` ở gốc cây là sổ những điều chưa biết theo tinh thần của Feynman: mỗi mục là một câu hỏi cụ thể mà tôi biết là mình chưa trả lời được. Nó chỉ có ích khi được thu ngắn dần, nên mỗi lần giải quyết xong một mục thì chuyển nó thành một đoạn trong bài chính và ghi ngày.

CẠM BÃY

Cây này hiện ở trạng thái **bản nháp có kiểm soát**, và cần biết rõ điều đó trước khi dựa vào nó.

`refs.bib` được soạn từ trí nhớ về các công trình gốc: tên tác giả, năm và venue đủ chính xác để tra cứu, nhưng *chưa* được đối chiếu với trang gốc của nhà xuất bản. Theo `learning-rules.md` mục 5, như vậy là chưa qua cửa kiểm chứng nào. Trước khi trích dẫn ra ngoài cây hoặc dùng để ra quyết định thật, phải mở bản gốc.

Quan trọng hơn với cây này: **các con số về độ chính xác góc** — kiểu “góc AprilTag khoảng 0,1–0,3 px, góc ChArUco sau tinh chỉnh khoảng 0,03–0,1 px” — là *bậc độ lớn mang tính kinh*

nghiệm, không phải kết quả đo của một thí nghiệm cụ thể có điều kiện xác định. Chúng đủ tốt để lập ngân sách sơ bộ và để so sánh tương đối giữa các phương pháp, và *không* đủ tốt để nghiệm thu. Con số σ của *hệ thống của bạn* phải do chính bạn đo, và chương 16 nói cách đo. Mọi chỗ trong cây dùng các con số này đều ghi rõ chúng là bậc độ lớn.

Câu hỏi còn mở (Open Questions)

Bốn điều cần xử lý khi quay lại cây này. Thứ nhất, toàn bộ refs.bib cần một lượt đối chiếu với nguồn gốc, và mỗi entry cần được dán mức tin A/B/C *sau khi* đã mở được ít nhất abstract. Thứ hai, script Monte Carlo ở C-ngan-sach-sai-so/16-danh-gia-va-nghiem-thu/code/ cần được chạy trên cấu hình thật và kết quả của nó phải được đối chiếu với công thức giải tích ở chương 5 — đây là cửa kiểm chứng (b) cho toàn bộ Phần C, và cho tới khi làm xong thì các công thức ở chương 5 mới chỉ qua cửa (a). Thứ ba, các con số σ góc cần được thay bằng số đo thật trên phần cứng cụ thể, kèm điều kiện đo. Thứ tư, Phần D chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất; rà lại mỗi sáu tháng.